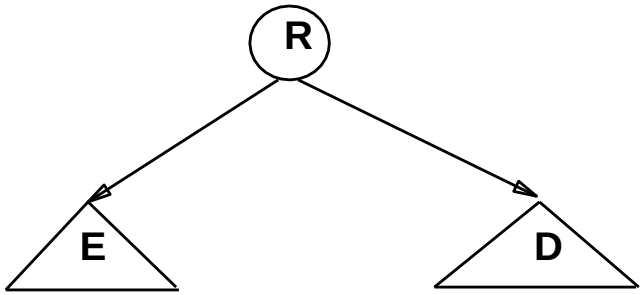
Árvores de Pesquisa

- A árvore de pesquisa é uma estrutura de dados muito eficiente para armazenar informação.
- Particularmente adequada quando existe necessidade de considerar todos ou alguma combinação de:
  1. Acesso direto e sequencial eficientes.
  2. Facilidade de inserção e retirada de registros.
  3. Boa taxa de utilização de memória.
  4. Utilização de memória primária e secundária.

Árvores Binárias de Pesquisa sem Balanceamento

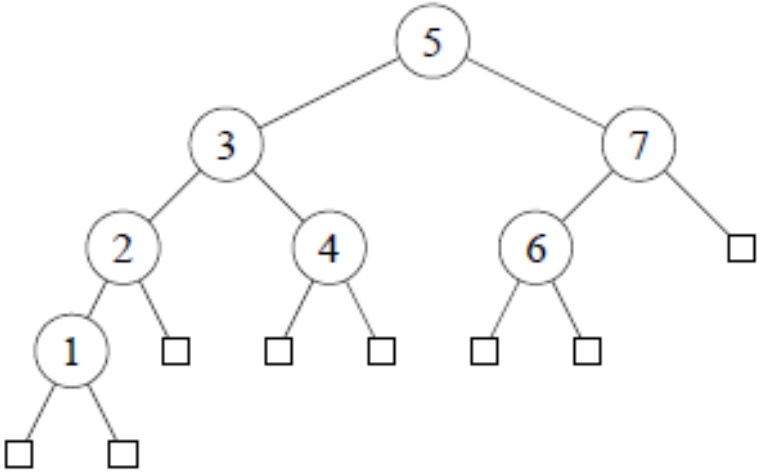
- Para qualquer nó que contenha um registro



Temos a relação invariante

1. Todos os registros com chaves menores estão na subárvore à esquerda.
2. Todos os registros com chaves maiores estão na subárvore à direita.

Árvores Binárias de Pesquisa sem Balanceamento



- O **nível** do nó raiz é 0.
- Se um nó está no nível i então a raiz de suas subárvores estão no nível $i + 1$.
- A **altura** de um nó é o comprimento do caminho mais longo deste nó até um nó folha.
- A altura de uma árvore é a altura do nó raiz.

Implementação do Tipo Abstrato de Dados Dicionário usando a Estrutura de Dados Árvore Binária de Pesquisa

Estrutura de dados:

```
typedef long TipoChave;
typedef struct TipoRegistro {
  TipoChave Chave;
  /* outros componentes */
} TipoRegistro;
typedef struct TipoNo * TipoApontador;
typedef struct TipoNo {
  TipoRegistro Reg;
  TipoApontador Esq, Dir;
} TipoNo;
```

Procedimento para Pesquisar na Árvore Uma Chave *x*

- Compare-a com a chave que está na raiz.
- Se *x* é menor, vá para a subárvore esquerda.
- Se *x* é maior, vá para a subárvore direita.
- Repita o processo recursivamente, até que a chave procurada seja encontrada ou um nó folha é atingido.
- Se a pesquisa tiver sucesso o conteúdo retorna no próprio registro *x*.

```
void Pesquisa(TipoRegistro *x, TipoApontador *p)
{ if (*p == NULL)
  { printf("Erro: Registro nao esta presente na arvore\n"); return; }
  if (x->Chave < (*p)->Reg.Chave)
  { Pesquisa(x, &(*p)->Esq); return; }
  if (x->Chave > (*p)->Reg.Chave) Pesquisa(x, &(*p)->Dir);
  else *x = (*p)->Reg;
}
```

Procedimento para Inserir na Árvore

- Atingir um apontador nulo em um processo de pesquisa significa uma pesquisa sem sucesso.
- O apontador nulo atingido é o ponto de inserção.

```
void Insere(TipoRegistro x, TipoApontador *p)
{ if (*p == NULL)
  { *p = (TipoApontador)malloc(sizeof(TipoNo));
    (*p)->Reg = x; (*p)->Esq = NULL; (*p)->Dir =
    NULL; return;
  }
  if (x.Chave < (*p)->Reg.Chave)
  { Insere(x, &(*p)->Esq); return;
  } if(x.Chave > (*p)->Reg.Chave)
  Insere(x, &(*p)->Dir);
  else printf("Erro : Registro ja existe na arvore\n");
}
```

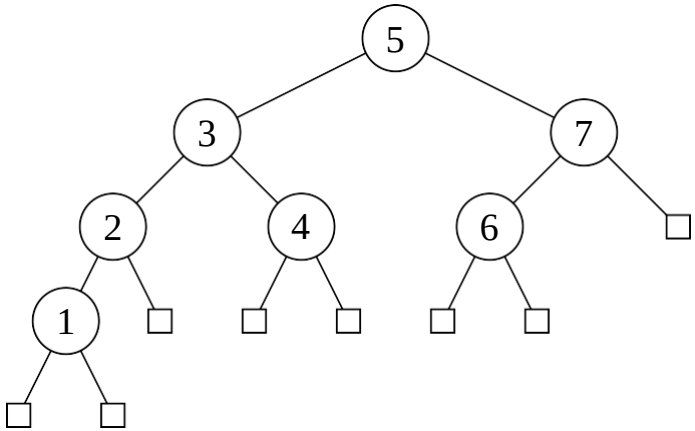
Procedimentos para Inicializar e Criar a Árvore

```
{-- Entra aqui a definição dos tipos --}

void Inicializa (TipoApontador *raiz)
{ *raiz = NULL; };


{-- Entram aqui os procedimentos Insere e Inicializa --}
int main( )
{ TipoApontador raiz; TipoRegistro x;
   Inicializa(&raiz);
   scanf("%d", &x.Chave);
   while(x.Chave > 0) {
         Insere(x,&raiz);
         scanf("%d", &x.Chave);
     }
}
```

Procedimento para Retirar x da Árvore

- Alguns comentários:
  1. A retirada de um registro não é tão simples quanto a inserção.
  2. Se o nó que contém o registro a ser retirado possui no máximo um descendente $\Rightarrow$ a operação é simples.
  3. No caso do nó conter dois descendentes o registro a ser retirado deve ser primeiro:
     - substituído pelo registro mais à direita na subárvore esquerda;
     - ou pelo registro mais à esquerda na subárvore direita.

Exemplo da Retirada de um Registro da Árvore



Assim: para retirar o registro com chave 5 na árvore basta trocá-lo pelo registro com chave 4 ou pelo registro com chave 6, e então retirar o nó que recebeu o registro com chave 5.

Procedimento para Retirar *x* da Árvore

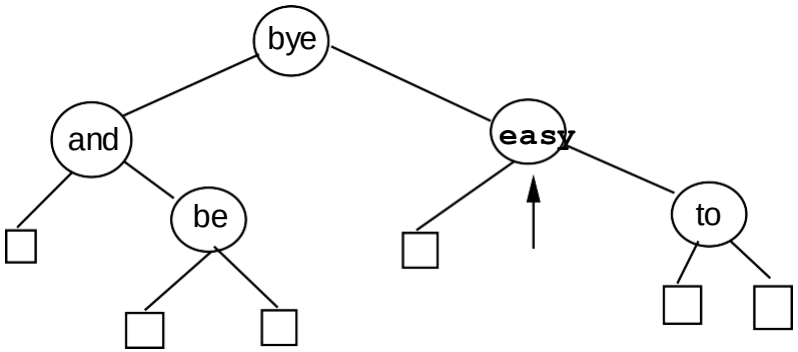
```
void Retira(TipoRegistro x, TipoApontador *p)
{ TipoApontador Aux;
  if (*p == NULL) { printf("Erro : Registro nao esta na arvore\n"); return; }
  if (x.Chave < (*p)->Reg.Chave) { Retira(x, &(*p)->Esq); return; }
  if (x.Chave > (*p)->Reg.Chave) { Retira(x, &(*p)->Dir); return; }
  if ((*p)->Dir == NULL)
  { Aux = *p;  *p = (*p)->Esq;
    free(Aux); return;
  }
  if ((*p)->Esq != NULL) { Antecessor(*p, &(*p)->Esq); return;}
  Aux = *p;  *p = (*p)->Dir;
  free(Aux);
}
```

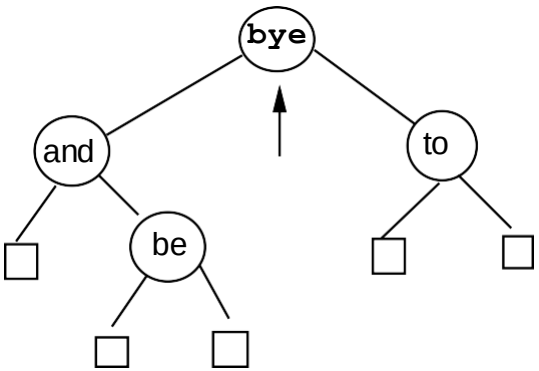
- **Obs.:** proc. recursivo Antecessor só é ativado quando o nó que contém registro a ser retirado possui 2 descendentes. Solução usada por Wirth, 1976, p.211.

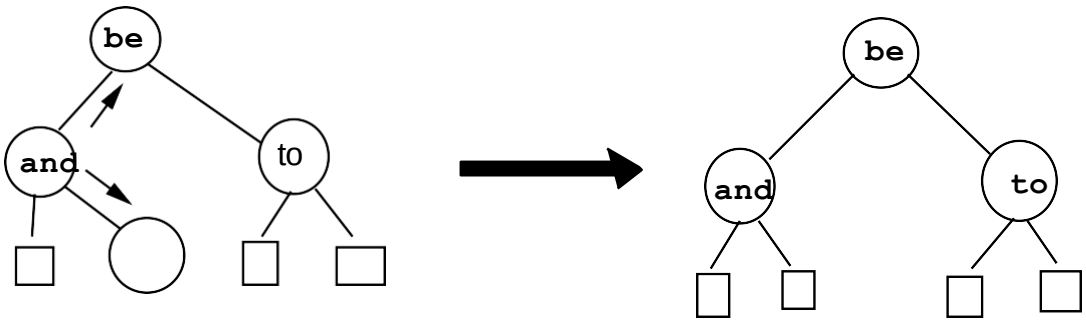
Procedimento para Retirar *x* da Árvore

```
void Antecessor(TipoApontador q, TipoApontador *r)
{ if ((*r)->Dir != NULL)
  { Antecessor(q, &(*r)->Dir);
    return;
  }
  q->Reg = (*r)->Reg;
  q = *r;
  *r = (*r)->Esq;
  free(q);
}
```

Outro Exemplo de Retirada de Nó







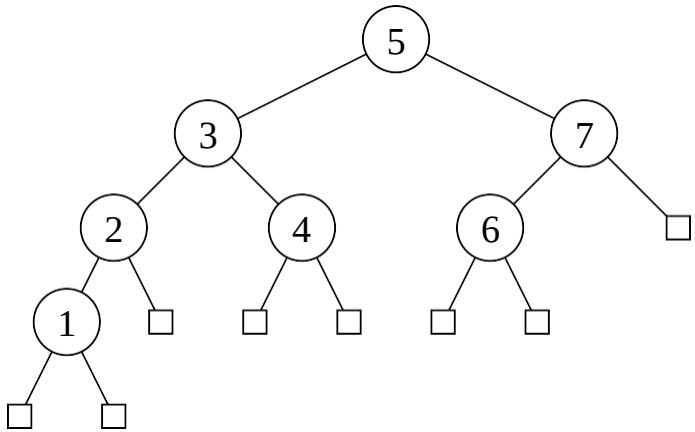
Caminhamento Central

- Após construída a árvore, pode ser necessário percorrer todos os registros que compõem a tabela ou arquivo.
- Existe mais de uma ordem de **caminhamento** em árvores, mas a mais útil é a chamada ordem de **caminhamento central**.
- O caminhamento central é mais bem expresso em termos recursivos:
  1. caminha na subárvore esquerda na ordem central;
  2. visita a raiz;
  3. caminha na subárvore direita na ordem central.
- Uma característica importante do caminhamento central é que os nós são visitados de forma ordenada.

Caminhamento Central

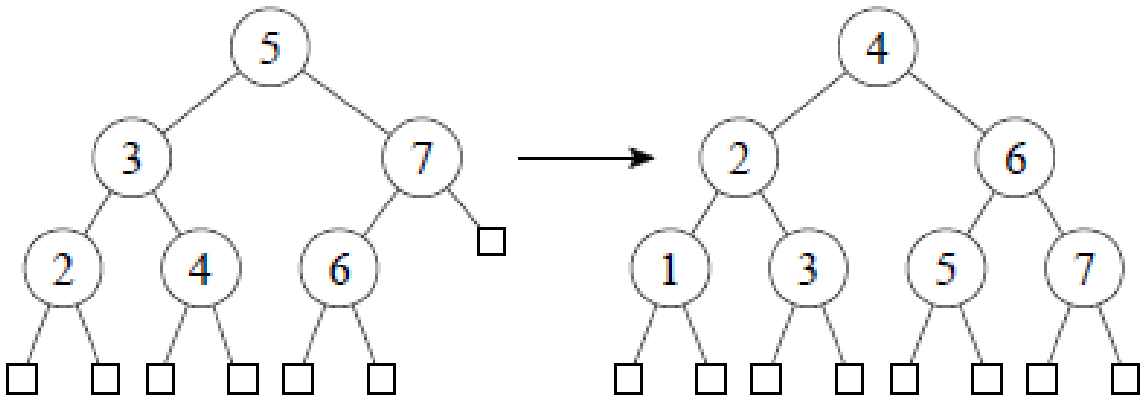
```
void Central(TipoApontador p)
{ if (p == NULL) return;
  Central(p->Esq);
  printf("%d\n", p->Reg.Chave);
  Central(p->Dir);
}
```

- Percorrer a árvore usando caminhamento central recupera, na ordem: 1, 2, 3, 4, 5, 6, 7.



Árvores Binárias de Pesquisa com Balanceamento

- Árvore completamente balanceada ⇒ nós externos aparecem em no máximo dois níveis adjacentes.
- Minimiza tempo médio de pesquisa para uma distribuição uniforme das chaves, onde cada chave é igualmente provável de ser usada em uma pesquisa.
- Contudo, custo para manter a árvore completamente balanceada após cada inserção é muito alto.
- Para inserir a chave 1 na árvore à esquerda e obter a árvore à direita é necessário movimentar todos os nós da árvore original.



Uma Forma de Contornar este Problema

- Procurar solução intermediária que possa manter árvore "quase-balanceada", em vez de tentar manter a árvore completamente balanceada.
- **Objetivo:** Procurar obter bons tempos de pesquisa, próximos do tempo ótimo da árvore completamente balanceada, mas sem pagar muito para inserir ou retirar da árvore.
- **Heurísticas:** existem várias heurísticas baseadas no princípio acima.
- Gonnet e Baeza-Yates (1991) apresentam algoritmos que utilizam vários critérios de balanceamento para árvores de pesquisa, tais como restrições impostas:
  - na diferença das alturas de subárvores de cada nó da árvore,
  - na redução do **comprimento do caminho interno**
  - ou que todos os nós externos apareçam no mesmo nível.